我用的debian

那更简单了！Debian本身就是apt的原生系统，直接在本机就能操作，不用WSL了，给你Debian下完整流程：

第一步：安装打包依赖

先更新源，装打包需要的工具：

```bash
sudo apt update
sudo apt install dpkg-dev gzip git build-essential
# 如果后续要上传到PPA，再加装这个
sudo apt install devscripts debmake
```

第二步：打包你的项目成 `.deb` 包

1. 整理项目，创建打包结构

假设你的项目叫 `mypkg`（就是用户 `apt install mypkg` 的名字），在你的项目目录下执行：

```bash
# 创建打包需要的目录结构
mkdir -p mypkg/usr/bin
mkdir -p mypkg/DEBIAN
```

- `mypkg/usr/bin`：放你的可执行程序/脚本
- `mypkg/DEBIAN`：放包的控制信息

2. 放入你的程序文件

把你编译好/写好的程序放进去，比如你是Python脚本：

```bash
```

```
# 把你的脚本复制进去，改名成你要的命令名
cp 你的项目脚本.py mypkg/usr/bin/mypkg
# 给执行权限，不然没法运行
chmod +x mypkg/usr/bin/mypkg
```

如果是编译好的二进制程序，直接替换 `你的项目脚本.py` 就行。

如果有额外的资源文件（比如配置、文档），就按系统路径放，比如要装到 `/usr/share/mypkg`，就创建 `mypkg/usr/share/mypkg`，把文件放进去就行。

3. 写包控制信息

创建 `DEBIAN/control` 文件，这个是apt识别包信息用的：

```bash
nano mypkg/DEBIAN/control
```

写入下面的内容，改成你自己的信息：

```
Package: mypkg          # 包名，用户apt install 后面跟的就是这个
Version: 1.0.0          # 版本号，更更新要改版本
Architecture: all       # 架构：所有架构通用写all，64位二进制写amd64，arm写arm64
Maintainer: 你的名字 <你的邮箱@xxx.com>
Description: 一句话描述你的包做什么
 这里写多行详细描述，换行开头必须加空格，说明功能就行
Depends: python3 (>=3.6) # 依赖，你的程序需要什么，apt会自动装，比如依赖python3就写这个，没有依赖就留空
```

写完按 `Ctrl+O` 回车保存，`Ctrl+X` 退出nano。

4. 打包生成deb

执行打包命令，一键生成deb：

```bash
dpkg-deb --build --root-owner-group mypkg
```

执行完当前目录就会生成 `mypkg.deb`，打包就完成了！

现在这个deb已经可以直接安装了，本地测试一下：

```bash
```

```
sudo dpkg -i mypkg.deb
# 测试能不能用：输入mypkg（你的包名），能运行就对了
```

第三步：做apt源（让用户可以apt install）

最省心免费的方式就是用GitHub Pages托管，步骤：

1. 创建源目录

```bash
# 创建源的目录结构
mkdir -p ~/my-apt-repo/pool/main/m/mypkg
mkdir -p ~/my-apt-repo/dists/stable/main/binary-amd64
# 把打好的deb复制进去
cp mypkg.deb ~/my-apt-repo/pool/main/m/mypkg/
```

2. 生成apt的索引文件

```bash
cd ~/my-apt-repo
dpkg-scanpackages pool/ /dev/null | gzip -9c > dists/stable/main/binary-amd64/Packages.gz
```

3. 推送到GitHub

1. 先去GitHub新建一个空仓库，比如叫 `my-apt-repo`，不要初始化README。
2. 回到Debian终端，执行：

```bash
cd ~/my-apt-repo
git init
git add .
git commit -m "init my apt repo"
# 绑定远程仓库，换成你的地址
git remote add origin git@github.com:你的用户名/my-apt-repo.git
git push -u origin main
```

3. 开启GitHub Pages： 打开GitHub这个仓库 → `Settings` → 左侧 `Pages` → Source选 `main` 分支，点保存，等1分钟就生效了，你的源地址就是： `https://你的用户名.github.io/my-apt-repo/`

最终用户安装方式

用户只需要在Debian/Ubuntu里执行三行：

```bash
# 添加你的源
echo "deb [trusted=yes] https://你的用户名.github.io/my-apt-repo/ stable main" | sudo tee /etc/apt/sources.list.d/mypkg.list
sudo apt update
sudo apt install mypkg
```

就完成安装了，完全就是你想要的 `apt install` 效果。

更简单的方案（适合个人小工具）

如果你不想维护源，直接把打好的 `mypkg.deb` 上传到GitHub项目的Release，用户也可以这样直接安装：

```bash
wget https://github.com/你的用户名/你的项目/releases/download/v1.0.0/mypkg.deb
sudo dpkg -i mypkg.deb
sudo apt install -f # 这一步会自动补全依赖，搞定依赖问题
```

比自己维护源简单很多，适合刚做包先测试。

哪一步不清楚随时问我~

(注:文档部分内容可能由AI生成)